



Programa Profesional en Full Stack Developer

Módulo 6. Despliegue y Testing de Aplicaciones

Índice

Tema 1. Despliegue en servicios de Hosting y Nube

- 1.1. Introducción y objetivos
- 1.2. Deploy de aplicación Angular en Firebase
- 1.3. Deploy de aplicación Angular en Netlify
- 1.4. Deploy de aplicación Express a Heroku

Tema 2. TDD

- 2.1. Introducción y objetivos
- 2.2. ¿Qué es TDD?
- 2.3. Testing en Angular con Jasmine
- 2.4. Testing en Express con Jest

Tema 3. Principios SOLID

- 3.1. Introducción y objetivos
- 3.2. Principios SOLID
- 3.3. Finalidad de los principios SOLID

Tema 4. CI/CD

- 4.1. Introducción y objetivos
- 4.2. ¿Qué es CI/CD?
- 4.3. Cómo podemos implementar CI/CD
- 4.4. ¿Por qué implementarla en nuestros proyectos?

A fondo

Cómo usar Testing en Angular con Jasmine y Karma

TDD (Test Driven Development) Desarrollo dirigido por pruebas

TDD | Test Driven Development en Api REST con Nodejs y Express desde cero

CleanCoder

Solid principles javascript

S.O.L.I.D The first 5 principles of Object Oriented Design with JavaScript

Comparación de integración continua, entrega e implementación continuas

Test

Tema 1. Despliegue en servicios de Hosting y Nube

1.1. Introducción y objetivos

La creación del software, tanto de la parte del cliente como de la parte del servidor, es solo una parte de todo el trabajo que debemos llevar a cabo cuando tenemos que crear una aplicación.

Antes de comenzar con el desarrollo una parte fundamental, por ejemplo, es la toma de requisitos, el análisis de experiencia de usuario y el diseño de interfaces. Son etapas que ayudan a que el desarrollo del código sea posteriormente mucho más eficiente y se aproxime a lo requerido por parte del cliente.

Una vez hemos terminado con el desarrollo la etapa más importante es la que nos permite desplegar nuestras aplicaciones dentro de algún servidor para que estén accesibles de manera online desde cualquier punto de la red.

En este tema vamos a trabajar con algunas plataformas que nos van a permitir subir nuestras aplicaciones, tanto de front como de back y además se van a encargar de mantenerlas activas para que no tengamos que depender de nuestros ordenadores para poder interactuar con ellas como hemos hecho hasta ahora.

Disponemos de muchos servicios de pago y otros servicios gratuitos que nos permiten realizar esta acción. Durante este tema analizaremos los más importantes.

Tema 1. Despliegue en servicios de Hosting y Nube

1.2. Deploy de aplicación Angular en Firebase

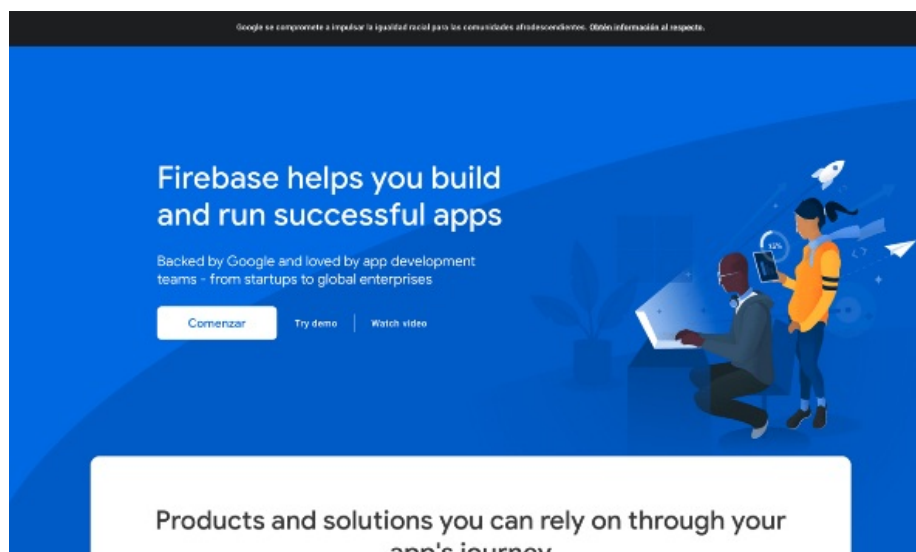
Firebase es una herramienta diseñada y mantenida por Google que nos permite subir nuestras aplicaciones y ponerlas online para que puedan ser consumidas desde cualquier navegador.

Además de actuar como espacio para subir nuestras aplicaciones, también dispone de una serie de herramientas que podemos utilizar si fuese necesario como la autenticación o el almacenamiento de imágenes y archivos estáticos.

Fue creada en 2011 y adquirida por Google en 2014. Desde entonces se ha convertido en uno de los servicios más utilizados independientemente de la tecnología que estemos usando. De entre todas las herramientas que tenemos disponibles destaca el servicio para la gestión de datos analíticos.

Para poder acceder a dichas herramientas tenemos que hacer login en <http://firebase.google.com>

Podemos entrar a través de una cuenta de correo de Google.



Tema 1. Despliegue en servicios de Hosting y Nube

Figura 1. Captura página web Firebase. Fuente: elaboración propia.

Dentro de la consola de Firebase <https://console.firebase.google.com>

Encontramos la posibilidad de crear un nuevo proyecto.

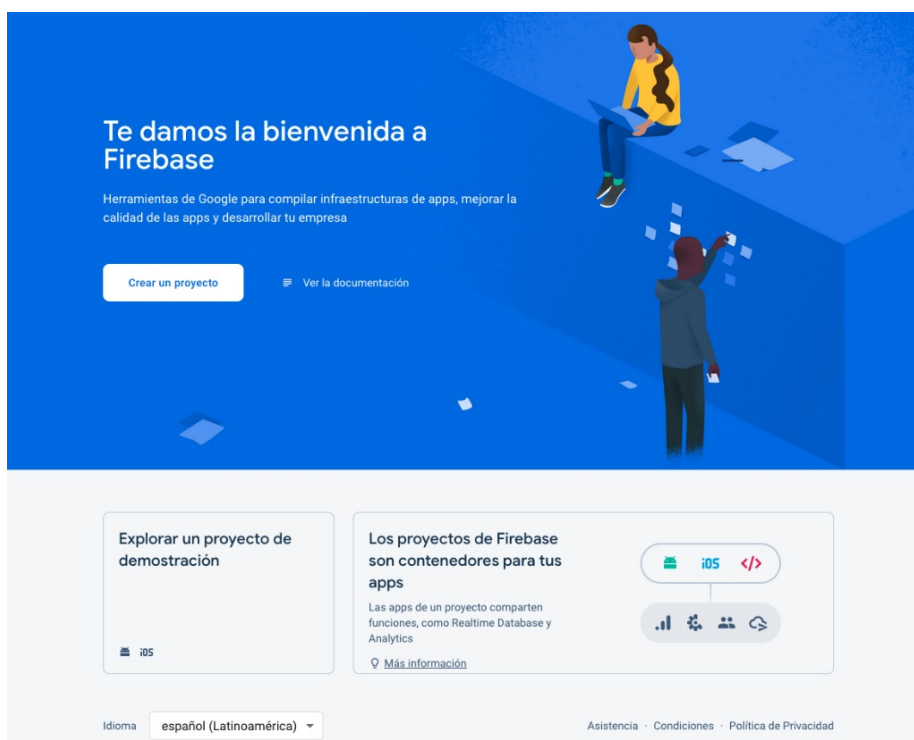


Figura 2. Captura página web Firebase. Fuente: elaboración propia.

Una vez pulsemos el botón de **Crear un proyecto** encontraremos el formulario que nos lleva a introducir los datos para la nueva aplicación.

En primera instancia, el nombre único de la aplicación.

Tema 1. Despliegue en servicios de Hosting y Nube



Figura 3. Captura página web Firebase – Creación de proyectos. Fuente: elaboración propia.

Como en este caso no vamos a trabajar con las analíticas de Firebase desactivamos el paso 2.

Tema 1. Despliegue en servicios de Hosting y Nube

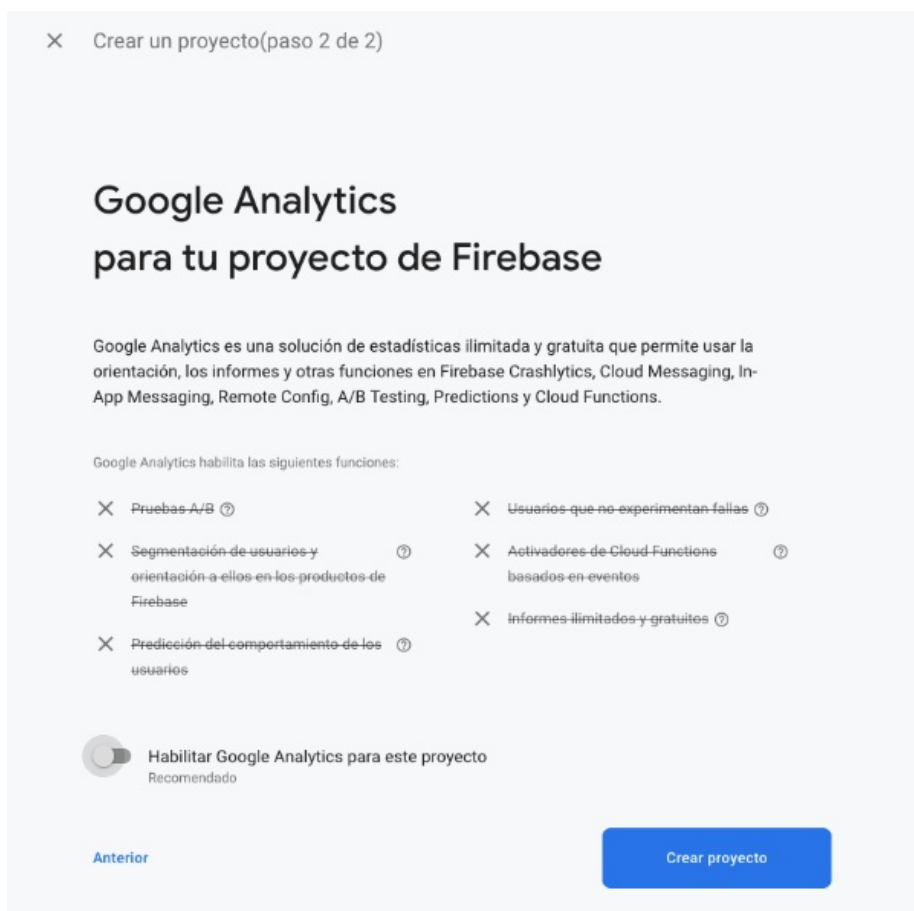


Figura 4. Captura página web Firebase – Creación de proyectos. Fuente: elaboración propia.

Y generamos el proyecto:

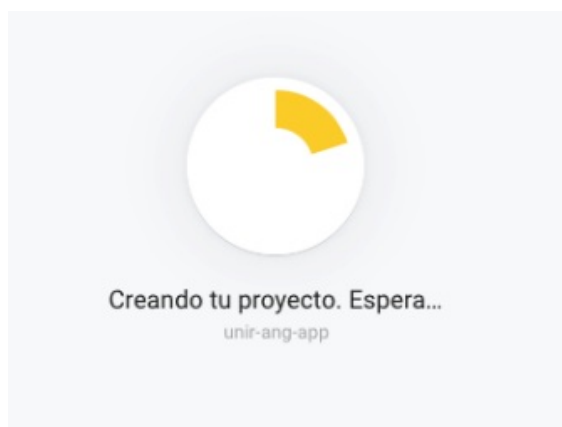


Figura 5. Captura página web Firebase – Creación de proyectos. Fuente: elaboración propia.

Tema 1. Despliegue en servicios de Hosting y Nube

A partir de este momento ya tenemos la aplicación creada en Firebase y podemos subir cualquier tipo de aplicación generada con Angular. Es importante saber que dentro de la parte de **hosting** de Firebase solo podemos incluir aplicaciones de tipo cliente. Para trabajar con aplicaciones del lado servido ya veremos una alternativa parecida.

Una vez tengamos seleccionada la aplicación de Angular que vamos a subir, lo primero que debemos hacer es instalar las herramientas de Firebase.

npm install -g firebase-tools

Esta ejecución a nivel global (-g) nos permite acceder a estas herramientas desde cualquier directorio de nuestra máquina local.

El primer paso es conectar nuestro ordenador con Firebase. Para ello podemos lanzar el siguiente comando y seguir las instrucciones pertinentes.

firebase login

Una vez tenemos la confirmación del login correcto, podemos pasar a enlazar la aplicación de Angular con su espacio dentro de Firebase. Para ello lanzamos el comando siguiente dentro del directorio de la aplicación.

firebase init

El comando nos realiza una serie de preguntas que debemos contestar de la siguiente manera:

Pregunta: Which Firebase features do you want to set up for this directory?

Respuesta: Hosting: Configure files for Firebase Hosting and (optionally) set up GitHub Action deploys

Pregunta: Please select an option

Tema 1. Despliegue en servicios de Hosting y Nube

Respuesta: Use an existing project

Seleccionamos el proyecto que hemos generado anteriormente dentro de Firebase

```
> unir-ang-app (unir-ang-app)
```

Figura 6. Captura Terminal. Fuente: elaboración propia.

Pregunta: What do you want to use as your public directory?

Respuesta: public

Pregunta: Configure as a single-page app (rewrite all urls to /index.html)?

Respuesta: yes

Pregunta: Set up automatic builds and deploys with GitHub?

Respuesta: No

El siguiente mensaje nos indica que la inicialización ha finalizado de manera correcta:

```
i Writing configuration info to firebase.json...
i Writing project information to .firebaserc...
✓ Firebase initialization complete!
```

Figura 7. Captura Terminal. Fuente: elaboración propia.

Una vez configurada la aplicación es turno de lanzar el comando que compila nuestra aplicación y la deja preparada para subir.

npm run build

Este comando nos genera un directorio **dist** con todos los ficheros necesarios para

Tema 1. Despliegue en servicios de Hosting y Nube

que nuestra aplicación pueda ser visualizada desde cualquier navegador.

Para que Firebase sepa dónde tiene que buscar todos estos ficheros en el momento de subirlos a la red, dentro del documento **firebase.json** debemos modificar la propiedad **public** y establecer correctamente el directorio.

```
{
  "hosting": {
    "public": "dist/PruebaNg",
    "ignore": [
      "firebase.json",
      "**/.*",

      "**/node_modules/**"
    ],
    "rewrites": [
      {
        "source": "**",
        "destination": "/index.html"
      }
    ]
  }
}
```

Una vez tengamos todos preparado podemos lanzar el comando que nos sube la aplicación a Firebase:

firebase deploy

Cuando termine la subida podemos ver como Firebase nos proporciona la URL donde está subida nuestra aplicación, utilizando el dominio **web.app**.

Tema 1. Despliegue en servicios de Hosting y Nube

```
=== Deploying to 'unir-ang-app'...

i  deploying hosting
i  hosting[unir-ang-app]: beginning deploy...
i  hosting[unir-ang-app]: found 7 files in dist/PruebaNg
✓  hosting[unir-ang-app]: file upload complete
i  hosting[unir-ang-app]: finalizing version...
✓  hosting[unir-ang-app]: version finalized
i  hosting[unir-ang-app]: releasing new version...
✓  hosting[unir-ang-app]: release complete

✓  Deploy complete!

Project Console: https://console.firebase.google.com/project/unir-ang-app/overview
Hosting URL: https://unir-ang-app.web.app
```

Figura 8. Captura Terminal. Fuente: elaboración propia

Si realizamos cambios sobre nuestra aplicación únicamente tendríamos que repetir los pasos que compilan la aplicación (`npm run build`) y el que sube los ficheros a Firebase (`firebase deploy`).

Tema 1. Despliegue en servicios de Hosting y Nube

1.3. Deploy de aplicación Angular en Netlify

Otro de los sistemas disponibles para subir nuestras aplicaciones de Angular es Netlify. Una de las ventajas grandes que vamos a poder observar con respecto a Firebase es que podemos enlazar su uso con un repositorio GIT y así gestionar de un mismo modo nuestro sistema de versiones con el hosting para la aplicación.

Debemos crearnos una cuenta dentro de la plataforma: <https://www.netlify.com/>

El primer paso que vamos a dar sobre una aplicación Angular es **subirla a un repositorio dentro de github.**

Podemos generar un repositorio específico dentro de github.com

Tema 1. Despliegue en servicios de Hosting y Nube

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Repository template
Start your repository with a template repository's contents.

No template ▾

Owner * **Repository name ***

 mariogiron ▾ / PruebaAngularNetlify ✓

Great repository names are short and memorable. Need inspiration? How about [miniature-waffle?](#)

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

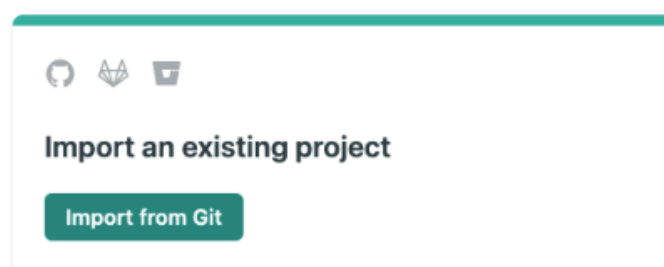
☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

Figura 9. Captura Github creación de repositorio. Fuente: elaboración propia

Seguimos las instrucciones para su creación y **hacemos push al repositorio generado.**

Dentro de la plataforma de Netlify seleccionamos la opción de importar un proyecto existente:



Tema 1. Despliegue en servicios de Hosting y Nube

Figura 10. Captura Netlify. Fuente: elaboración propia.

El primer paso consiste en conectar y dar permisos a la plataforma de donde cogeremos el código. En este caso, github.

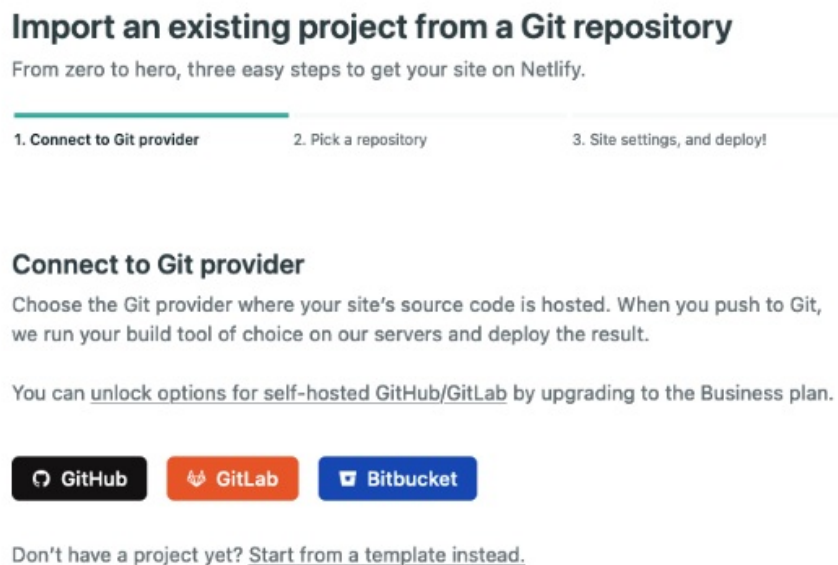


Figura 11. Captura Netlify. Fuente: elaboración propia.

En el segundo paso seleccionamos el repositorio donde hemos subido el código de la aplicación Angular.

Tema 1. Despliegue en servicios de Hosting y Nube

Import an existing project from a Git repository

From zero to hero, three easy steps to get your site on Netlify.

1. Connect to Git provider

2. Pick a repository

3. Site settings, and deploy!

Pick a repository from GitHub

Choose the repository you want to link to your site on Netlify. When you push to Git, we run your build tool of choice on our servers and deploy the result.

 mariogiron ▾

Q Search repos

 mariogiron/PruebaAngularNetlify

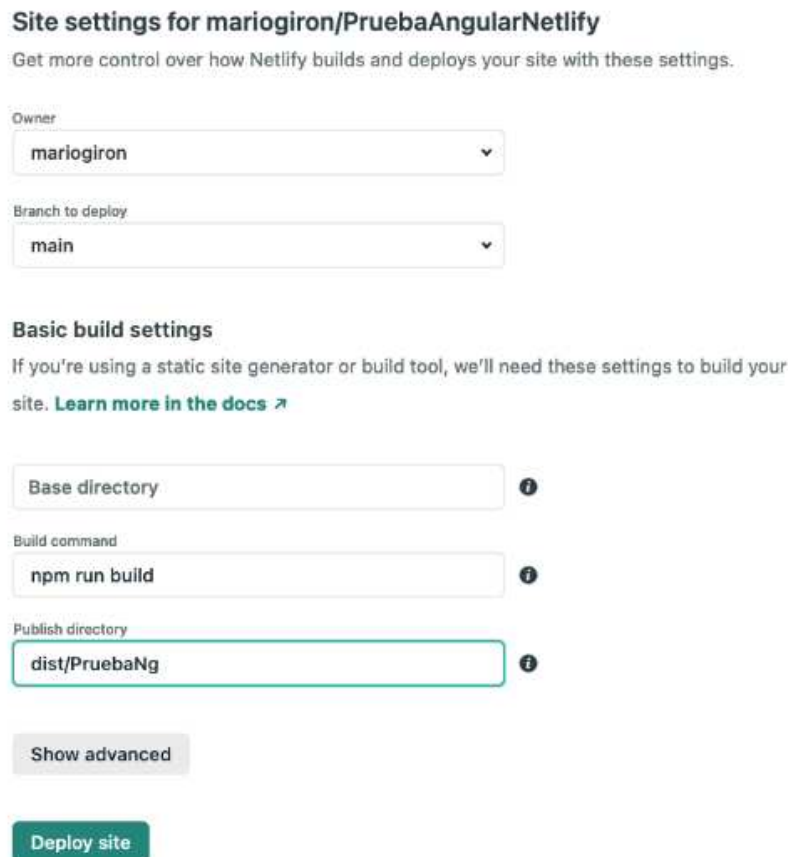


Can't see your repo here? [Configure the Netlify app on GitHub.](#)

Figura 12. Captura Netlify. Fuente: elaboración propia.

Por último, definimos la rama donde se encuentra el código, así como las configuraciones básicas para compilar la aplicación y concretar el directorio donde encontrar todos los ficheros compilados.

Tema 1. Despliegue en servicios de Hosting y Nube



Site settings for mariogiron/PruebaAngularNetlify
Get more control over how Netlify builds and deploys your site with these settings.

Owner
mariogiron

Branch to deploy
main

Basic build settings
If you're using a static site generator or build tool, we'll need these settings to build your site. [Learn more in the docs](#)

Base directory

Build command
npm run build

Publish directory
dist/PruebaNg

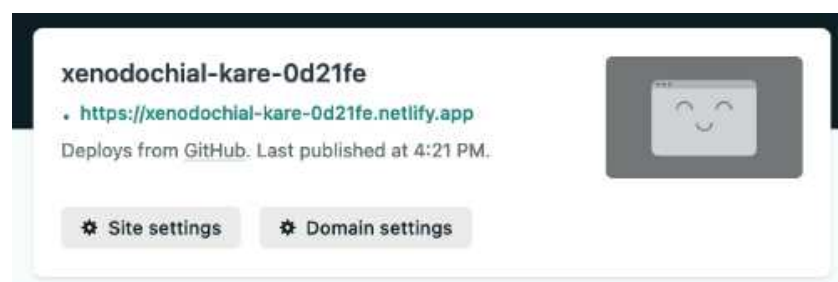
Show advanced

Deploy site

Figura 13. Captura Netlify. Fuente: elaboración propia.

Pulsando el botón de **Deploy site** comenzará a funcionar el sistema interno de Netlify para recoger todos los datos de nuestra aplicación, compilarla y ponerla a funcionar en producción.

Una vez terminen todas las acciones asociadas al deploy obtendremos la URL asignada a nuestra aplicación.

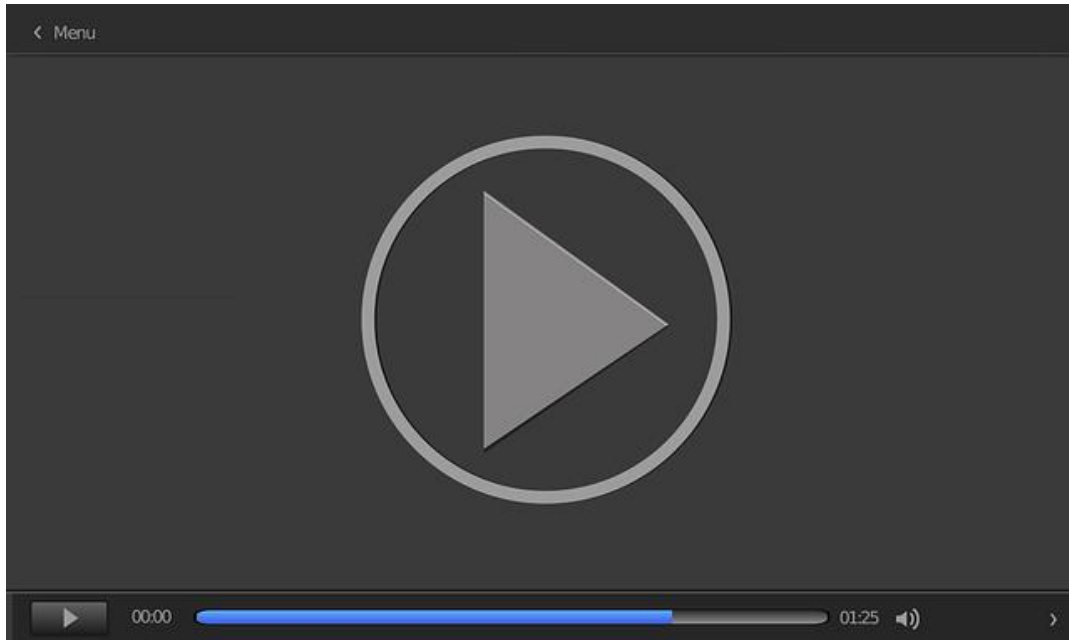


Tema 1. Despliegue en servicios de Hosting y Nube

Figura 14. Captura Netlify. Fuente: elaboración propia.

Tema 1. Despliegue en servicios de Hosting y Nube

En este vídeo, **Desplegando en Netlify**, veremos cómo hacer un despliegue de una aplicación de una aplicación en Angular en Netlify y todos sus pasos detenidamente.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=bebfacff-6362-4641-8b06-ae57014170a8>

Tema 1. Despliegue en servicios de Hosting y Nube

1.4. Deploy de aplicación Express a Heroku

Las dos opciones anteriores incluyen herramientas para poder colocar online nuestras aplicaciones de Front. Se trata de sistemas con no demasiados requerimientos ya que el final, lo único que deben hacer es servidor código HTML, CSS y Javascript.

En el caso de nuestras aplicaciones del back, se complica un poco más la búsqueda de plataformas que puedan cumplir con los requisitos mínimos. El espacio donde subamos nuestras aplicaciones de Express debe disponer de un sistema que nos permita ejecutar NodeJS y que se encargue de tener nuestra aplicación constantemente atenta a las diferentes peticiones entrantes.

Para solventar esta situación contamos con **Heroku**, una plataforma que nos permite, a través de un repositorio GIT subir nuestras aplicaciones y se encarga automáticamente de gestionar todo el tema de la instalación de dependencias y el mantenimiento de la aplicación para que constantemente esté respondiendo a las peticiones entrantes.

El primer paso para subir nuestras aplicaciones es darnos de alta en la plataforma: <https://heroku.com>

Una vez tengamos acceso a la página podemos crear nuestra aplicación con un nombre único.

Tema 1. Despliegue en servicios de Hosting y Nube

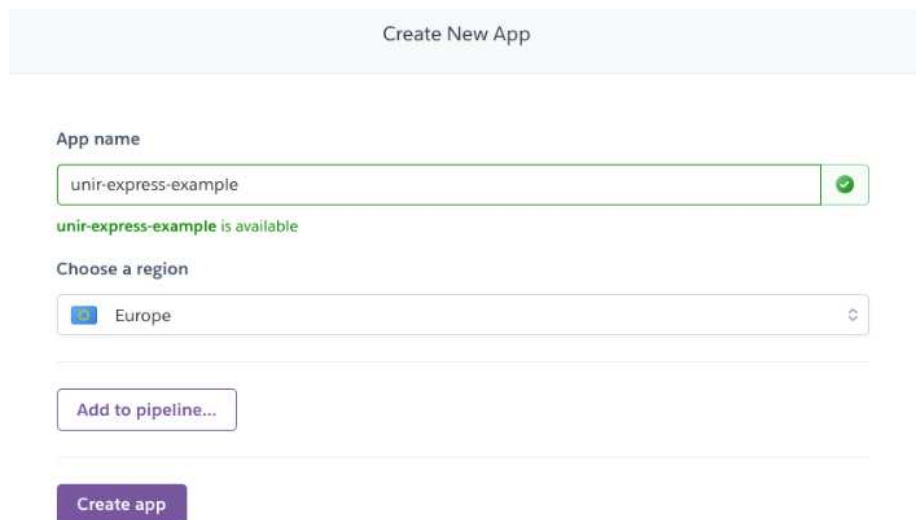


Figura 15. Captura Heroku. Fuente: elaboración propia.

Desde la sección para la instalación de las herramientas de Heroku podemos descargarnos la que corresponda a nuestro sistema operativo.

<https://devcenter.heroku.com/articles/heroku-cli#download-and-install>

Una vez instalada podemos hacer login desde la consola a través del siguiente comando.

heroku login

A partir de ese punto únicamente queda seguir las instrucciones que tenemos disponibles dentro de la sección **deploy** de la aplicación creada recientemente.

Tema 1. Despliegue en servicios de Hosting y Nube

Create a new Git repository

Initialize a git repository in a new or existing directory

```
$ cd my-project/  
$ git init  
$ heroku git:remote -a unir-express-example
```

Deploy your application

Commit your code to the repository and deploy it to Heroku using Git.

```
$ git add .  
$ git commit -am "make it better"  
$ git push heroku master
```

Figura 16. Captura Heroku. Fuente: elaboración propia.

Cuando hacemos push sobre el nuevo repositorio remoto, heroku automáticamente se encarga de subir nuestra aplicación, instalar las dependencias y determinar qué acciones son necesarias en el momento de la instalación.

Además, una vez termine obtendremos la url con nuestra aplicación de Express correctamente funcionando online.

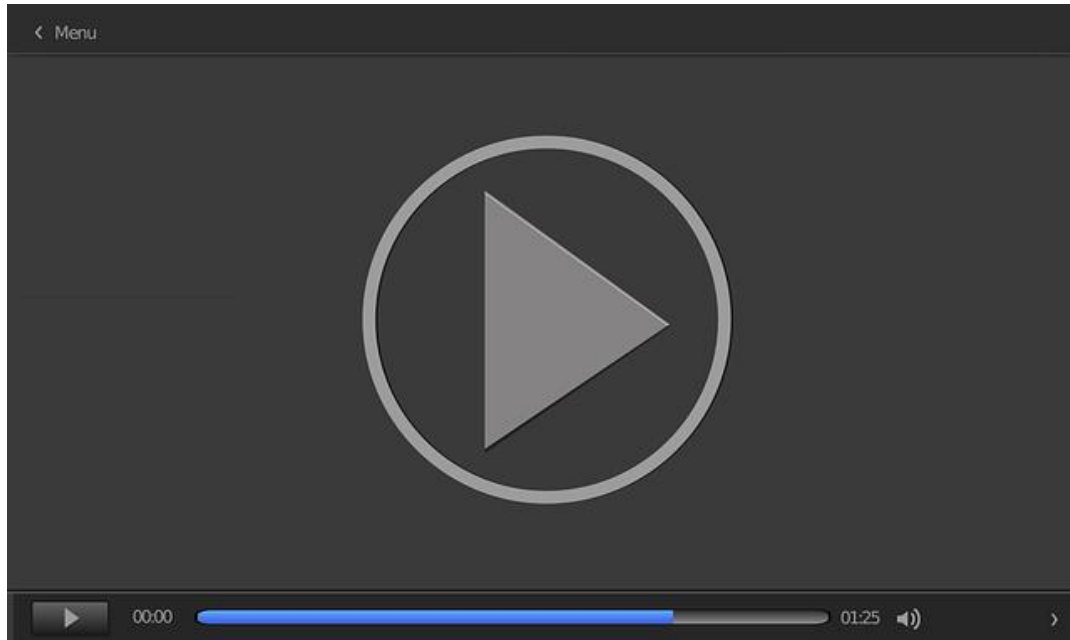
Heroku es una plataforma que incluye todos los servicios necesarios para que una aplicación de estas características funcione. Dispone de un servicio gratuito, que es el que estamos usando nosotros como nuevos usuarios, pero también ofrece la opción de hacer crecer nuestras aplicaciones con su versión de pago.

Por si fuera poco, nos ofrece una serie de herramientas adicionales que podemos utilizar dentro de nuestras aplicaciones, donde se incluyen bases de datos o sistemas de logging.

<https://elements.heroku.com/addons>

Tema 1. Despliegue en servicios de Hosting y Nube

En este vídeo, **Desplegando en Heroku**, veremos cómo hacer un despliegue de una aplicación en Express dentro de heroku y todos sus pasos detenidamente.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=4bf82139-6a81-4404-b8ce-ae57013de97f>

2.1. Introducción y objetivos

Es muy fácil que los desarrolladores cuando nos enfrentamos a un proyecto en concreto «nos engañemos a nosotros mismos». Es decir, cuando estamos probando el código que desarrollamos muchas veces pasamos por alto muchos aspectos que nos pueden llevar a obtener errores en etapas posteriores de la implementación del proyecto.

Es muy importante diseñar y desarrollar una serie de pruebas que nos permitan tener control sobre los posibles errores que estemos generando en la etapa de desarrollo de nuestras aplicaciones.

En muchas ocasiones incluso existe un equipo específico para el desarrollo de pruebas que recibe nuestro código y se encarga de pasarlo por diferentes test para comprobar cuál es su rendimiento y cómo se pueden mejorar ciertos aspectos.

Esto no quita que nosotros, a nivel individual no destinemos cierto tiempo de nuestro desarrollo a crear estas pruebas que nos van a proporcionar un código mucho más robusto y con menos errores en el futuro.

En este tema vamos a establecer una base para comenzar en el mundo de las pruebas tanto en la parte del front como en la parte del back, determinando una serie de buenas prácticas con los frameworks de pruebas más utilizados.

2.2. ¿Qué es TDD?

Haciendo un resumen muy condensado, TDD (**test driven development**) es la práctica mediante la cual planteamos nuestros desarrollos de software centrándonos primero en el desarrollo de las pruebas y posteriormente en el propio código.

Se trata de llevar a cabo una serie de técnicas repetitivas que combinen aspectos de programación, la creación de pruebas unitarias y la posterior refactorización del código.

El código que surge de seguir de manera correcta y ordenada las siguientes pautas es un código mucho más limpio, flexible y escalable, lo que hace que el desarrollador pueda trabajar de manera más cómoda a la hora de implementar nuevas funcionalidades o dar soporte a las que ya están creadas.

Los desarrolladores empiezan generando pequeños bloques de pruebas ya que uno de los principios fundamentales sobre el TDD es que **únicamente debemos escribir código final si una prueba ha fallado**.

Podemos resumir estas prácticas en 3 fases totalmente diferenciadas:

- ▶ **Creación de test:** Debemos ser capaces de desarrollar tests que verifiquen cada una de las nuevas funcionalidades de nuestra aplicación. Sabiendo cuales son los requisitos previos y teniendo experiencia con la herramienta elegida para las pruebas, el desarrollador debe poder generar estas pruebas con anterioridad al código.
- ▶ **Corrección del código:** Una vez comprobamos que el test falla se debe escribir el mínimo código requerido para que el test pase correctamente. Para ello en muchas ocasiones es imprescindible escribir más de una prueba para cada funcionalidad, dependiendo de los diferentes resultados que pueda generar.

- **Refactorización del código:** Cuando la prueba se ejecute de manera satisfactoria es el momento de buscar cómo podemos modificar y mejorar el código escrito. Debemos prestar atención a código repetido y posibles mejoras. Esta refactorización del código debe hacerse sin romper ninguna de las funcionalidades previas.

Un esquema resumen de las fases anteriores podría ser el siguiente:

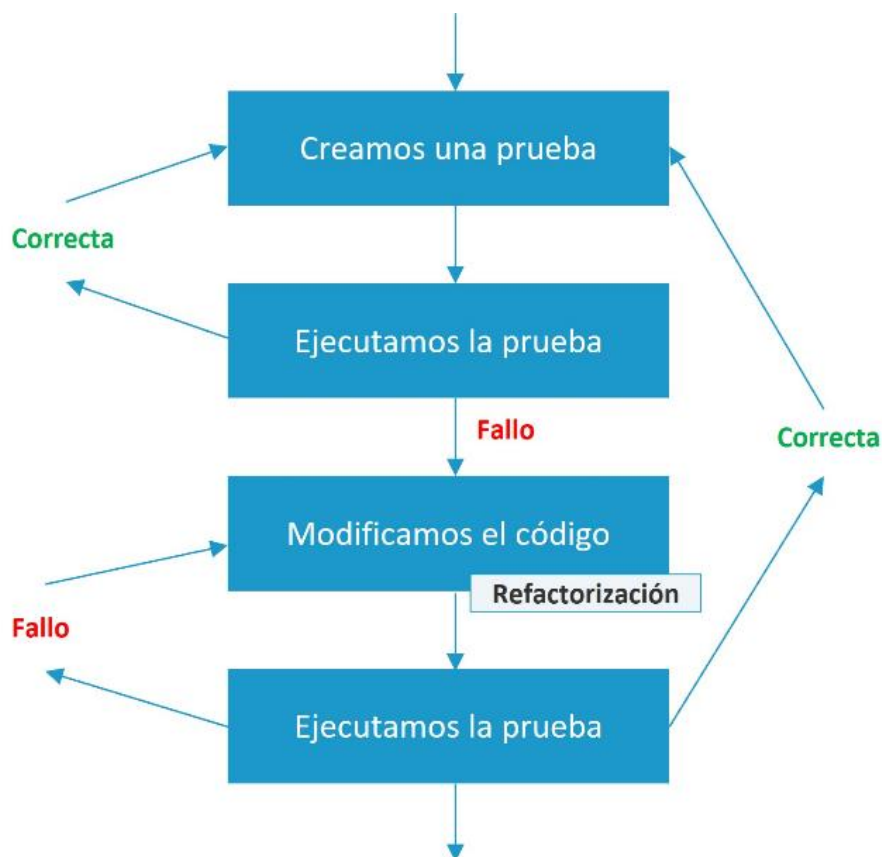


Figura 17. Explicación del flujo de un test. Fuente: elaboración propia

Beneficios del TDD

Vamos a destacar alguno de los beneficios que nos aporta seguir esta serie de prácticas sobre nuestros diferentes proyectos de desarrollo.

- ▶ Se genera un **código muy optimizado**. Únicamente vamos a desarrollar aquel código necesario para poder pasar las pruebas que planteemos, por lo que nos vamos a evitar mucho código sobrante.
- ▶ **Es de gran ayuda para que los desarrolladores comprendan los requerimientos del cliente**. Las pruebas hay que desarrollarlas en función de los objetivos que planteemos para la aplicación, por lo que como desarrolladores debemos comprender y analizar qué pide el cliente y si lo que hemos recogido como requisitos están correctos.
- ▶ Si desarrollamos correctamente una estrategia de pruebas, **la creación de nuevas funcionalidades según avancen las etapas del proyecto serán más sencillas**.
- ▶ **Todo lo que desarrollemos dentro nuestras aplicaciones estará probado**. Esto nos quita mucho tiempo y trabajo en fases posteriores.
- ▶ **Mejora la productividad de los desarrolladores y mejora sus habilidades de cara a la creación de código limpio y mantenible**.

Existen multitud de frameworks de pruebas que nos permiten llevar a cabo esta estrategia. En este tema veremos los más importantes para la implementación de pruebas con Angular y con ExpressJS.

2.3. Testing en Angular con Jasmine

Se trata de un framework que ya incluye todas las herramientas necesarias para poder testear nuestro código. Podemos lanzarlos e incluso poder generar reportes.

Podemos generar ficheros de tests para diferentes elementos dentro de nuestras aplicaciones.

La nomenclatura habitual es ponerle el mismo nombre del fichero que vamos a testear, pero con la extensión **spec.ts** para así diferenciarlos. Esta extensión le sirve al framework para poder detectar en qué ficheros se encuentran las pruebas y en cada caso ejecutarlas.

Test Suite

La estructura de nuestros ficheros de pruebas parte de un **Test Suite** y dentro del mismo tendremos todas las especificaciones que vamos a probar. Partimos de una llamada al método **describe** donde concretamos cómo se llama la Suite y qué función contendrá el resto de las especificaciones.

Vamos a probar un servicio que contendrá una serie de métodos para realizar cálculos matemáticos. Generamos el fichero **calculator.service.spec.ts**

```
describe('CalculatorService', () => {  
});
```

Cada una de las especificaciones que generemos tienen como objetivo probar la funcionalidad de ciertas partes de nuestro código. Dicho de otra manera, lo usamos para probar las diferentes ejecuciones de una función dentro de nuestras aplicaciones.

Podemos definir una prueba a partir del método **it**. Podemos incluir todas las pruebas

que necesitemos dentro de una misma suite:

```
describe('CalculatorService', () => {  
  it('debería sumar dos números', () => {  
  });  
  it('debería restar dos números', () => {  
  });  
});
```

Dentro de nuestras pruebas, para indicar a Jasmine que la prueba no está implementada todavía podemos llamar al método **pending**.

```
describe('CalculatorService', () => {  
  it('debería sumar dos números', () => {  
    pending();  
  });  
  it('debería restar dos números', () => {  
    pending();  
  });  
});
```

Podemos lanzar las pruebas con el comando **ng test** y comprobar cómo se lanzan todas marcando de amarillo las que se encuentran pendientes.

Karma es la herramienta que se encarga de la ejecución y reporte de los tests y cada vez que los ejecutamos obtenemos el resultado dentro de un navegador que se abre automáticamente.

La ejecución con Karma es constante y cada vez que hacemos un cambio se encarga de compilar la aplicación y volver a ejecutar las pruebas activas. Podemos probarlo lanzando el método **fail** en alguna de las pruebas anteriores.

Si queremos lanzar las pruebas sin esta característica podemos ejecutar el comando **ng test —no-watch**

Test sencillo

Para poder empezar a probar las funciones que tenemos dentro de nuestros servicios, lo primero que tenemos que hacer es disponer de una instancia de dicho servicio que nos permita ejecutar lo necesario.

Mediante el método **expect** acompañado de uno de los métodos que comprueba su resultado, podemos especificar qué es lo que necesitamos probar.

```
import { CalculatorService } from './calculator.service';
describe('CalculatorService', () => {
  it('debería sumar dos números', () => {
    const calculator = new CalculatorService();
    const result = calculator.add(2, 3);
    expect(result).toBe(4);
  });
  it('debería restar dos números', () => {
    pending();
  });
});
```

Para poder generar esta prueba debemos generar el servicio **calculator.service.ts**.

```
import { Injectable } from '@angular/core';
@Injectable({
  providedIn: 'root'
})
export class CalculatorService {
  constructor() { }
  add(a: number, b: number): number {
    return a * b;
  }
}
```

De esta manera, lanzando el grupo de pruebas podemos ver que pasan todas ya que se cumplen con las expectativas planteadas.

Podemos crear la prueba para el método subtract

```
it('debería restar dos números', () => {
  const calculator = new CalculatorService();
  const result = calculator.subtract(3, 2);
  expect(result).toBe(1);
});
```

```
});  
beforeEach
```

Mediante este método podemos inicializar todos aquellos objetos que vayamos a utilizar en más de un caso de prueba y así nos ahorramos tener que estar repitiendo tanto código y generando tantas variables que en el fondo son lo mismo.

Este método se ejecuta antes de cada uno de los casos de test que vayamos a definir dentro de una Suite.

Trabajando con este método, las pruebas anteriores quedarían de la siguiente manera.

```
import { CalculatorService } from './calculator.service';  
describe('CalculatorService', () => {  
  let calculator: CalculatorService;  
  beforeEach(() => {  
    calculator = new CalculatorService();  
  });  
  it('debería sumar dos números', () => {  
    const result = calculator.add(2, 3);  
    expect(result).toBe(5);  
  });  
  it('debería restar dos números', () => {  
    const result = calculator.subtract(3, 2);  
    expect(result).toBe(1);  
  });  
});
```

De esta manera solo gestionamos una instancia del servicio, por lo que nos ahorramos tener que generar un objeto por cada una de las pruebas que lancemos. Debemos tener en cuenta que el propio método **beforeEach** genera su propio ámbito por lo que, si generamos ahí dentro cualquier tipo de variable, no la vamos a poder utilizar posteriormente.

TestBed – Inyección de dependencias

Mediante la funcionalidad de TestBed vamos a poder pasar dependencias a nuestras pruebas sin necesidad de crear instancias a partir de sus constructores. Nos permite utilizar la misma técnica que usamos en Angular para gestionar este tipo de objetos.

```
import { TestBed } from "@angular/core/testing";
import { CalculatorService } from "../calculator.service";
describe('CalculatorService', () => {
  let calculator: CalculatorService;
  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [CalculatorService]
    });
    calculator = TestBed.inject(CalculatorService);
  });
  it('debería sumar dos números', () => {
    const result = calculator.add(2, 3);
    expect(result).toBe(5);
  });
  it('debería restar dos números', () => {
    const result = calculator.subtract(3, 2);
    expect(result).toBe(1);
  });
});
```

Lo que hacemos en el código anterior es recuperar la instancia del servicio CalculatorService a través de la llamada al método **configureTestingModule**, por lo que ya no estamos generando una instancia de dicho servicio si no que estamos dejando que Angular la genere por nosotros.

De todas estas acciones previas sacamos varias conclusiones:

- ▶ Todos los tests deben estar aislados entre si. Por eso utilizamos herramientas como el método **beforeEach** que nos permite reinicializar las instancias que estamos usando para cada test.
- ▶ Únicamente nos deberíamos traer aquellos servicios que estemos probando, ya que así estaríamos haciendo verdaderas **Pruebas Unitarias** en vez de Pruebas de integración.

- Es importante usar la inyección de dependencias en nuestros tests ya que así es como trabaja todo el framework.

Utilidades Jasmine

Si queremos deshabilitar un grupo de pruebas, en vez de llamar al método `describe`, utilizamos el método **`xdescribe`**.

```
xdescribe('CalculatorService', () => {
```

Lo mismo pasa si queremos bloquear la ejecución de alguna prueba en concreto. Utilizamos el método **`xit`** para ello.

```
xit('debería sumar dos números', () => {
```

Jasmine Spies

Se puede presentar el caso en el que, dentro del servicio que estamos probando, necesitemos usar otro servicio asociado para realizar ciertas tareas dentro del primero. Dentro del código de Angular únicamente implicaría inyectar el nuevo servicio dentro del constructor del servicio original.

En este ejemplo generamos un servicio **`LoggerService`** el cual usamos para mostrar ciertos mensajes dentro del servicio que estamos probando.

```
import { Injectable } from '@angular/core';
@Injectable({
  providedIn: 'root'
})
export class LoggerService {
  constructor() { }
  log(msg: string) {
    console.log(`[MENSAJE] ${msg}`);
  }
}
```

```
}
import { Injectable } from '@angular/core';
import { LoggerService } from './logger.service';
@Injectable({
  providedIn: 'root'
})
export class CalculatorService {
  constructor(private logger: LoggerService) { }
  add(a: number, b: number): number {
    this.logger.log('Realizamos la SUMA');
    return a + b;
  }
  subtract(a: number, b: number): number {
    this.logger.log('Realizamos la RESTA');
    return a - b;
  }
}
```

Podemos, a través de los **spies de Jasmine** analizar el número de veces que se ejecuta este método y convertirlo en una prueba. Dentro del método **beforeEach** del fichero de pruebas del servicio de la calculadora, incluimos también la inyección del nuevo servicio.

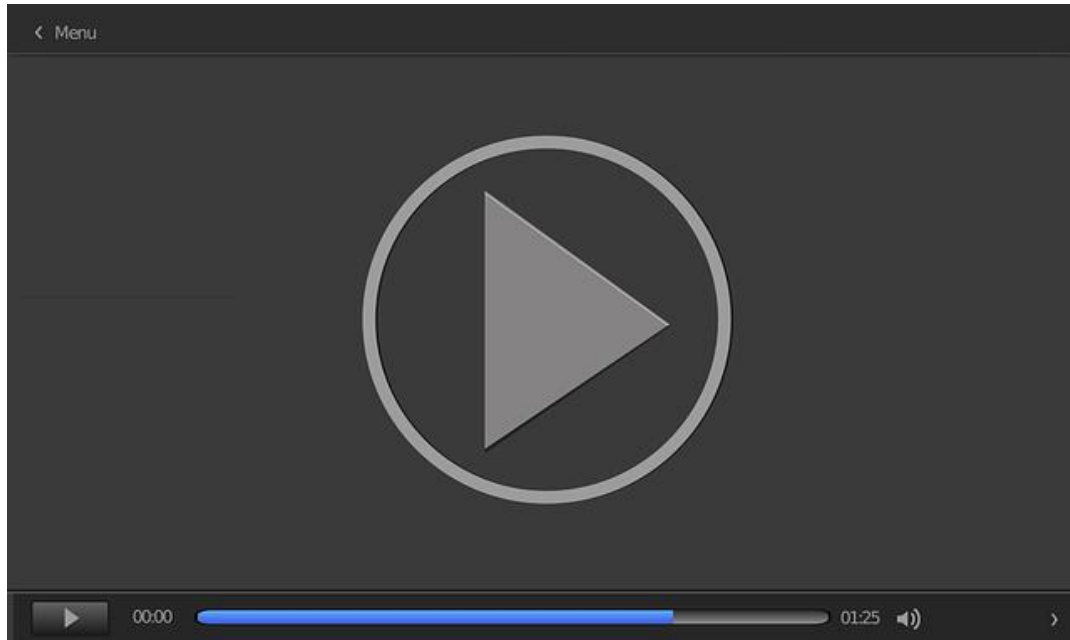
```
beforeEach(() => {
  loggerSpy = jasmine.createSpyObj('LoggerService', ['log']);
  TestBed.configureTestingModule({
    providers: [
      CalculatorService,
      { provide: LoggerService, useValue: loggerSpy }
    ]
  });
  calculator = TestBed.inject(CalculatorService);
});
```

Dentro de los diferentes casos de prueba podemos utilizar este objeto **loggerSpy** para determinar si se ejecuta el número correcto de veces.

```
it('debería sumar dos números', () => {
  const result = calculator.add(2, 3);
  expect(result).toBe(5);
  expect(loggerSpy.log).toHaveBeenCalledTimes(1);
});
```

Tema 2. TDD

En el vídeo **Testing en un App de Angular** vamos a realizar unos test dentro de algunos componentes para explicar el proceso dentro de una aplicación en Angular.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7fc1630a-f5ba-48d6-a82f-ae5f00b72174>

2.4. Testing en Express con Jest

Los mismos conceptos que hemos estado analizando para el desarrollo de pruebas con Angular lo podemos aplicar cuando necesitemos realizar nuestras comprobaciones previas sobre una aplicación de back generada a través de ExpressJS.

En este caso el cambio más llamativo es que aquí vamos a utilizar un framework diferente para poder llevar a cabo el desarrollo, Jest.

Lo bueno de esta librería es que no necesitamos ningún tipo de configuración previa y la podemos utilizar directamente dentro de cualquier proyecto basado en Javascript siguiendo siempre las mismas prácticas. Genera tests aislados del código y reutilizables. Además, vamos a ver cómo los mismos métodos que hemos utilizado en la sección anterior, también los encontramos dentro de Jest.

Preparación. Proyecto de prueba

Para poder definir una serie de pruebas, vamos a generar una aplicación sencilla de Express que responde a las siguientes rutas:

GET <http://localhost:3000/api/people>

POST <http://localhost:3000/api/people>

Esta aplicación se puede encontrar en el siguiente repositorio de Github.

<https://github.com/mariogiron/unir-testapp>

Una vez descargada debemos tener en cuenta una serie de cosas:

- ▶ Necesitamos una instancia de MongoDB arrancada en el ordenador
- ▶ Tenemos que lanzar **npm install** antes de realizar cualquier acción.
- ▶ Dentro del fichero **peticiones.rest** tenemos una serie de pruebas que podemos utilizar para comprobar que todo funciona correctamente.

Definición de pruebas

Antes de poder definir las pruebas debemos instalar las dependencias necesarias para que JEST funcione dentro de nuestro proyecto.

npm install jest @types/jest --save-dev

Con este comando estamos instalando la librería y además los tipos relacionados con la misma para que nuestro editor de código entienda los métodos y propiedades que vamos a utilizar.

Además, incluimos el flag **--save-dev** para indicar que estas dependencias solo se deben instalar si estamos en el entorno de desarrollo, que es donde vamos a ejecutarlas.

Una vez instalado, para poder ejecutar nuestras pruebas podemos definir el script **test** dentro del fichero **package.json**.

```
"scripts": {  
  "start": "node ./bin/server",  
  "dev": "nodemon ./bin/server",  
  "test": "jest"  
},
```

Si lanzamos **npm test** estaríamos lanzando las pruebas de nuestra aplicación.

Para poder especificar una serie de pruebas que se encarguen de comprobar todas las funcionalidades de las rutas que hemos generado, dentro del directorio tests

podemos crear el fichero **people.spec.js**. Cuando Jest detecte esa extensión ejecutará todos los casos de prueba que coloquemos dentro del nuevo fichero generado.

Una estructura inicial podría ser la siguiente:

```
describe('Peticones People', () => {  
  describe('GET /api/people', () => { })  
  describe('POST /api/people', () => { })  
});
```

Como hemos visto en la sección anterior, los diferentes casos de pruebas se encapsulan en llamadas al método **describe**. Para el caso que estamos tratando ahora definimos un grupo de pruebas para todas las peticiones de la API y posteriormente un grupo de pruebas independiente para cada una de las peticiones.

Una acción previa que debemos llevar a cabo en nuestro caso es pensar qué ámbito vamos a probar. La parte importante de la API se encuentra dentro de **app.js**. Pero hay que tener en cuenta que desde este fichero no se realiza ningún tipo de conexión con la base de datos.

Tenemos que procurar esta conexión antes de que empiecen nuestras pruebas y cerrarla cuando terminen. Esto podemos realizarlo con la implementación de los métodos **beforeAll** y **afterAll**, los cuales se ejecutarán antes de todas las pruebas y después respectivamente.

```
const mongoose = require('mongoose');  
describe('Peticones People', () => {  
  beforeAll(async () => {  
    await mongoose.connect('mongodb://127.0.0.1/unir-test');  
  });  
  afterAll(async () => {  
    await mongoose.disconnect();  
  });  
  describe('GET /api/people', () => { })  
  describe('POST /api/people', () => { })  
});
```

En nuestro caso, lo importante es probar si las peticiones que hemos desarrollado se realizan de manera correcta y nos devuelven los datos que estamos esperando. Para ello debemos simular la misma acción que haría el cliente realizando la petición con el método correcto y la url seleccionada.

Para realizar esta acción tenemos una librería muy sencilla de utilizar llamada **supertest**.

npm install supertest --save-dev

Ayudándonos de las herramientas que nos ofrece supertest, para cada uno de los grupos de pruebas específicos podemos lanzar la petición correspondiente. Como necesitaremos lanzar la petición delante de cada una de nuestras pruebas podemos utilizar el método **beforeEach** de la siguiente manera.

```
const mongoose = require('mongoose');
const request = require('supertest');
const app = require('../app');
describe('Peticiones People', () => {
  beforeAll(async () => {
    await mongoose.connect('mongodb://127.0.0.1/unir-test');
  });
  afterAll(async () => {
    await mongoose.disconnect();
  });
  describe('GET /api/people', () => {
    let response;
    beforeEach(async () => {
      response = await request(app).get('/api/people').send();
    })
  })
  describe('POST /api/people', () => { })
});
```

A partir de esta acción ya podemos empezar a comprobar cómo responde nuestro servidor a las diferentes peticiones.

Vamos a establecer una serie de pruebas para la petición que recupera todas las

personas comprobando lo siguiente:

- ▶ Que la respuesta es correcta, es decir, que nos devuelve el estado 200.
- ▶ Que el contenido devuelto en la respuesta es de tipo JSON.
- ▶ Que recibimos un array con datos.

La sintaxis es muy parecida a la vista antes, a partir del método **it** podemos lanzar suposiciones gracias a la ejecución de **expect** y las diferentes funciones de comprobación.

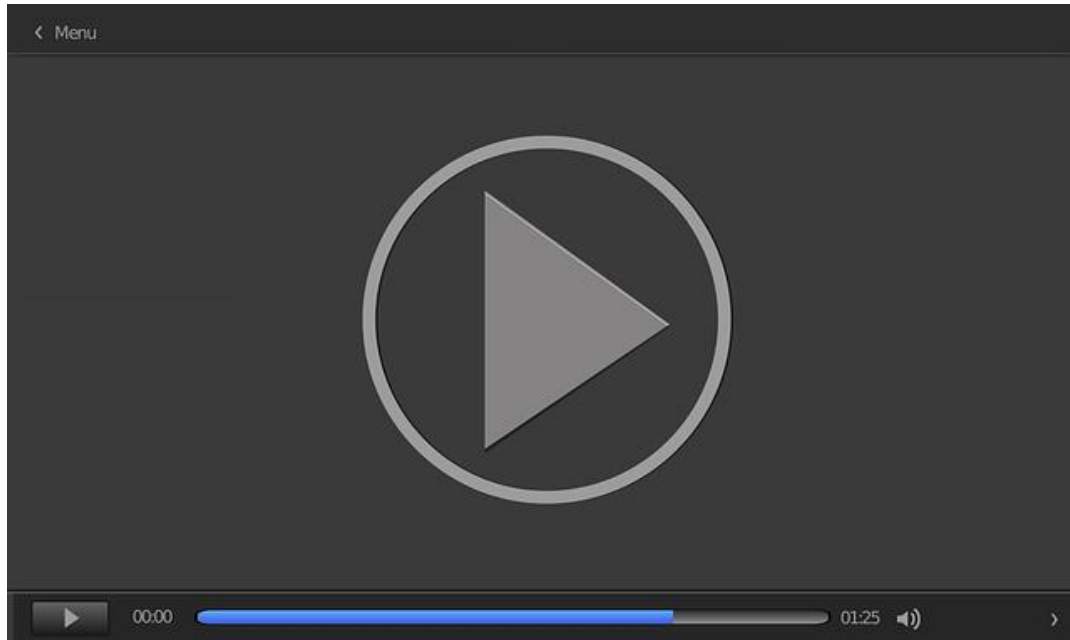
```
describe('GET /api/people', () => {
  let response;
  beforeEach(async () => {
    response = await request(app).get('/api/people').send();
  })
  it('debería devolver estado y contenido correcto', () => {
    expect(response.statusCode).toBe(200);
    expect(response.headers['content-type']).toContain('json');
  });
  it('debería devolver un array con datos', () => {
    expect(response.body).toBeInstanceOf(Array);
    // El id depende de nuestra base de datos
    expect(response.body[0]._id).toBe('61f10d397407a7da8034cc7b');
  });
})
```

De esta manera tan sencilla podríamos comprobar cada uno de los requerimientos que nos planteemos para las rutas de nuestra API.

Para la ruta de creación de personas deberíamos tener en cuenta una cosa importante. **Nuestras pruebas deben dejar la base de datos tal y como se la encontraron.** Es decir, cuando terminen las pruebas debemos borrar todo lo que hemos introducido o actualizado.

Las pruebas serían prácticamente iguales a las anteriores, comprobando los aspectos básicos de la respuesta.

En el vídeo **Testing en una aplicación de Express** vamos a explicar de forma más visual como se realizan técnicas de test a una aplicación backend.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=431226de-b705-4cab-a201-ae5f00a289ac>

3.1. Introducción y objetivos

En este tema vamos a tratar como concepto teórico adaptado a la programación que significa los principios SOLID y que ventajas trae aplicarlos dentro de nuestro código.

SOLID es el acrónimo que fue introducido por Robert C. Martin a principios del siglo XXI que representa los 5 principios que se deben tener en cuenta cuando realizas programación orientada a objetos.

Estos principios SOLID son una serie de recomendaciones que puedes decidir aplicar o no en tus desarrollos pero que te permiten crear software, en este caso web, escalable y extensible y con un desarrollo de código que sigue los estándares del código limpio.

La palabra SOLID está formada por la primera letra de cada uno de los cinco principios.

Los objetivos de SOLID son:

- ▶ Intentar que nuestro código sea más mantenible.
- ▶ Que sea fácil aplicar cambios y arreglar errores.
- ▶ Facilitar la incorporación de nuevas funcionalidades.
- ▶ Hacer el código limpio y fácil de entender.

3.2. Principios SOLID

Los 5 principios SOLID son los siguientes:

Los nombres están en Ingles para mantener el acrónimo de SOLID.

- ▶ Single Responsibility.
- ▶ Open Closed.
- ▶ Liskov Substitution.
- ▶ Interface Segregation.
- ▶ Dependency Inversión.

Vamos a intentar explicar un poco más en detalle cada uno de estos principios y entender cómo funciona. Intentaremos mostrar un ejemplo para que vayáis viendo en que consiste cada principio y como se aplica. Y así poder entender como nos pueden ayudar a escribir mejor código.

Principio de Responsabilidad Única (Single Responsibility)

Este principio indica que cada clase debería tener solo una única responsabilidad, es decir debería encargarse de una sola parte de nuestro desarrollo.

El objetivo es conseguir que nuestras clases hagan una sola cosa. El principio dicta de la siguiente manera.

«Nunca debería haber más de un motivo por el cual cambiar una clase o un módulo». Tener más de una responsabilidad dentro de nuestras clases o módulos hace que nuestro código sea difícil de leer, testear y mantener.

Tema 3. Principios SOLID

Es bastante común que los programadores apliquemos mal este principio ya que solemos pensar que tener una única responsabilidad es hacer una única cosa y no es así.

¿Qué es tener una única responsabilidad?

Ante todo, no consiste en crear clases que tenga único método, si no en diseñar componentes que sólo estén enfocados en realizar una cosa como por ejemplo registrar un usuario dentro de una plataforma.

Dicha acción puede ser llevada a cabo por la combinación de diferentes métodos y propiedades y el gran objetivo es por ejemplo separar la creación del usuario de la generación de la contraseña encriptada.

Vamos a ver un ejemplo incorrecto para que se puede entender estos conceptos. Tenemos el siguiente código.

```
class User {  
  createUser(alias, pass) {  
    let salt = bcrypt.genSaltSync(10)  
    let passEncrypted = bcrypt.hashSync(pass, salt)  
    const newUser = new User(alias, passEncrypted)  
    return newUser;  
  }  
}
```

En el ejemplo que vemos anterior, nuestra clase tiene doble responsabilidad, la de crear un usuario que recibe como parámetro y la de encriptar la contraseña. De tal modo que, si hay que modificar cualquiera de las dos acciones ya sea el concepto de crear un nuevo usuario o cambiar por ejemplo el sistema de encriptado, la realización de ambos cambios afecta a la totalidad de la clase y de su funcionalidad.

Veamos ahora como hubiese sido un planteamiento más correcto separando responsabilidades y aplicación el principio de responsabilidad única. Debemos poner por un lado la creación de usuario en una clase y la funcionalidad de encriptación a

otra.

```
class User {
  createUser(alias, pass) {
    let passEncrypted = PassEncrypter.encrypt(pass)
    const newUser = new User(alias, passEncrypted)
    return newUser;
  }
}

class PassEncrypter {
  static encrypt(pass) {
    let salt = bcrypt.genSaltSync(10);
    let encrypted = bcrypt.hashSync(pass, salt)
    return encrypted;
  }
}
```

En el caso de los códigos anteriores podemos ver que hemos separado en dos clases las dos funcionalidades de nuestro código separando su funcionalidad y responsabilidad.

La clase que genera el usuario delega a la clase de encriptado la responsabilidad de crear el password.

Principio de Abierto/Cerrado (Open Closed)

«Todas las entidades de software deben quedarse abiertas para su posible extensión, pero cerradas a modificación»

El objetivo de este principio nos recomienda que en los casos en los que tengamos que añadir nueva funcionalidad o comportamientos en nuestro sistema, no modifiquemos las funcionalidades antiguas si no que creamos unas nuevas.

La razón de esto es que, si modificamos una clase y esta se está usando dentro de otro componente, sin saberlo estamos modificando también el comportamiento de dicho componente provocando efectos indeseados.

Tema 3. Principios SOLID

Tenemos que intentar escribir código que no se tenga que cambiar cada vez que se necesites cambiar los requerimientos del sistema.

Aunque podemos pensar que en algunos aspectos este principio se contradice, existen varias técnicas que podemos aprender para aplicarlo. Algunas de las más importantes las podemos encontrar en la POO como son la herencia y el polimorfismo.

Imaginemos una clase que me defina el área de un cuadrado. Por ejemplo:

```
class Square {  
  constructor(alto, ancho) {  
    this.alto = alto;  
    this.ancho = ancho;  
  }  
}
```

Ahora queremos calcular el Área total de varios cuadrados así que creamos una clase que nos permita realizar esta funcionalidad.

```
class CalculadoraAreas{  
  getTotalArea(cuadrados){  
    let area = 0;  
    for (let cuadrado of cuadrados ){  
      area += (cuadrado.alto * cuadrado.ancho)  
    }  
    return area  
  }  
}
```

Hasta aquí todo correcto, pero y si necesitamos añadir la funcionalidad de calcular el área de otro tipo de polígono que no sea un cuadrado por ejemplo un triángulo.

Si se diese ese caso entonces deberíamos condicionar la suma del área en función de la que queramos calcular si la del cuadrado o la del círculo. Modificando así la funcionalidad del CalculadorAreas.

```
Class CalculadoraAreas {
```

Tema 3. Principios SOLID

```
getTotalArea(poligonos) {
  let area = 0;
  for (let poligono of poligonos) {
    if (typeof poligono === 'cuadrado') {
      area += (poligono.alto * poligono.ancho)
    } else {
      area += (poligono.alto * poligono.ancho) / 2
    }
  }
}
```

En el ejemplo de código que vemos, hemos modificado la funcionalidad de nuestra calculadora de áreas para incluir la del triángulo, el problema es que si en el futuro tenemos que calcular más áreas tendremos que seguir modificando la clase con lo que ello supone.

De esta forma nuestra clase está abierta a cambios de funcionalidad con lo que también está abierta a poder registrar nuevos errores.

La forma correcta de implementar esto sería de la siguiente forma. Deberíamos aplicar un polimorfismo, en este caso la clase Square debería calcular su propia área para así el calculador de áreas no tenga que preguntarse que tipo de objeto es simplemente pedirle al objeto que devuelva su área.

De tal forma que añadir nuevos objetos sería tan fácil como crear nuevas clases

```
class Square {
  constructor(alto, ancho) {
    this.alto = alto;
    this.ancho = ancho;
  }
  getArea(){
    return this.alto * this.ancho
  }
}
```

De esta forma el calculador de áreas solo tiene que llamar al método `getArea()` para obtener el valor de dicha área.

```
class CalculadoraAreas {
  getTotalArea(poligonos) {
    let area = 0;
```

Tema 3. Principios SOLID

```
for (let poligono of poligonos) {  
  area += poligono.getArea()  
}  
return area;  
}  
}
```

El resultado obtenido es un diseño de aplicación mucho más robusto y que nos facilita mucho a la hora de extenderlo ya que solo debemos crear nuevas clases que sigan el mismo patrón.

Principio de sustitución de Liskov (Liskov Substitution)

«Las funciones que utilicen referencias a otras clases padre o base deben ser capaces de usar objetos de clases derivadas sin saberlo»

Simplificando un poco el principio, este viene a decir que toda clase que sea hija de otra clase debe usarse de la misma forma que si fuese la clase padre.

En el contexto de JavaScript, esto significa que:

- ▶ Cada argumento del método sobrescrito debe ser del mismo tipo que el método de la clase base.
- ▶ El tipo de retorno del método sobrescrito debe ser el mismo que el método de la clase base.
- ▶ Los tipos de excepciones lanzadas desde el método anulado deben ser los mismos que el método de la clase base.

Vamos a mostrar un ejemplo para entender este principio y lo que representa.

```
class Bird {  
  fly(speed) {  
    return `Volando a ${speed} km/h`;  
  }  
}
```


Tema 3. Principios SOLID

```
}  
class Eagle extends Bird {  
  attack() {  
    return 'En picado a por comida'  
  }  
  fly(speed) {  
    return `Surcando los cielos a ${speed} km/h`;  
  }  
}  
  
<em>// LSP Error:  
</em>class Penguin extends Bird {  
  fly() {  
    return new Error("Sorry, I cant fly");  
  }  
}
```

En este ejemplo, la clase Eagle anula el método fly, esto no viola el principio de sustitución porque la creación de los nuevos métodos es compatible con el método de la clase base.

Los métodos de una subclase que anulan los métodos de una clase base deben tener el mismo número de argumentos. El principio de sustitución no insiste en que el valor de retorno de un método anulado también tenga que ser el mismo.

La clase Penguin viola el principio de sustitución de tres maneras:

- ▶ El método fly invalidado no tiene el mismo número de argumentos.
- ▶ El tipo de retorno del método fly no es el mismo.
- ▶ Los tipos de excepciones lanzadas no son las mismas.

En javascript una posible solución sería implementar una clase Animal que implementará los métodos attack y move dado que un Penguin también se mueve de manera diferente en este caso a un Bird, el movimiento resultante podría ser volar o caminar siempre y cuando la salida sea del mismo tipo. Con esta clase Animal Bird y Penguin podrían heredar de ella sin violar este principio.

```
class Animal {
```

Tema 3. Principios SOLID

```
move(speed) {  
  return `Volando a ${speed} km/h`;  
}  
attack() {  
  return `Iniciando el ataque`;  
}  
}  
class Bird extends Animal {  
  attack() {  
    return 'En picado a por comida'  
  }  
  move(speed) {  
    return `Surcando los cielos a ${speed} km/h`;  
  }  
}  
class Penguin extends Bird {  
  move() {  
    return `Surcando los mares a ${speed} km/h`;  
  }  
}
```

Principio de Segregación de la interfaz (Interface Segregation)

Este principio con un nombre muy barroco y poco entendible, en el que uno puede pensar hasta que significa la palabra segregación vamos a intentar explicarlo de la forma más fácil y entendible posible.

Este principio lo que nos viene a aconsejar es que apliquemos el famoso término «**divide y vencerás**» es decir es mejor tener funcionalidades más pequeñas que realicen cosas pequeñas por lo tanto tengan errores pequeños.

Separar responsabilidades va un poco ligado con todo lo que hemos estado viendo en este curso, básicamente no podemos crear una función que haga todo lo que teníamos pensado realizar porque es muy difícil de mantener, muy costosa su actualización y puede incluso ralentizar nuestra aplicación.

Este principio argumenta que es mejor hacer funciones especializadas que una función enorme que lo haga todo dentro de la aplicación.

Tema 3. Principios SOLID

Esto a su vez tiene ventajas con respecto al reparto de tareas dentro de un equipo de trabajo ya que si es posible separar responsabilidades dentro de nuestro código también es posible que dichas responsabilidades puedan ser repartidas dentro de un equipo de trabajo.

El tener separadas las funcionalidades también te permite que tu aplicación sea más escalable ya que para añadir nuevas funcionalidades solo hay que crear nuevas acciones y hacer mínimos cambios dentro de nuestra aplicación principal.

Veamos un ejemplo muy sencillo pero que ilustra muy bien lo que estamos diciendo.

```
function calcular(numero1, numero2, tipo) {
  let resultado = 0;
  switch (tipo) {
    case 'suma':
      <em>//sumar
    </em>resultado = sumar(numero1, numero2)
      break;
    case 'resta':
      <em>//restar
    </em>resultado = restar(numero1, numero2)
      break;
    case 'multiplicar':
      <em>//multiplicar
    </em>resultado = multiplicar(numero1, numero2)
      break;
    default:
      console.log('operacion no valida')
      break;
  }
  return resultado;
}

function sumar(pNumero1, pNumero2) {
  let resultado = pNumero1 + pNumero2;
  return resultado;
}

function restar(pNumero1, pNumero2) {
  let resta = pNumero1 - pNumero2;
  return resta;
}

function multiplicar(pNumero1, pNumero2) {
  let multiplicacion = pNumero1 * pNumero2;
```

Tema 3. Principios SOLID

```
return multiplicacion;  
}
```

Este ejemplo quizás pueda parecer muy sencillo, pero ilustra perfectamente lo que es este principio.

Si una analizar la primera función puede llegar a pensar por que llama a otras funciones dentro de la función principal. Cuando podría hacer perfectamente los cálculos dentro de la estructura condicional.

Pues la respuesta es sencilla, ya que este ejemplo puede parecer muy fácil de implementar en una sola función, pero lo realmente difícil es darse cuenta de que muchas veces nuestro código hace muchas más cosas de las que debe y que podemos repartir nuestro trabajo en otros elementos más pequeños y ser igualmente funcionales.

Cuan fácil sería implementar la función dividir en este ejemplo sin la necesidad de cambiar ni la entrada ni la salida de nuestras funciones y así no incumplir ningún principio anterior.

Lo difícil de este principio no es verlo aquí, si no en programas más complicados que nos puede llevar a confusión.

Principio de Inversión de Dependencia (Dependency Inversion)

El principio de inversión de dependencias se refiere a la estabilidad y la capacidad de reutilización de los componentes de alto nivel dentro de una aplicación. El principio establece:

«Los módulos de alto nivel no deberían depender de los módulos de bajo nivel»

El objetivo principal de este principio es que los componentes principales de mi aplicación permanezcan desacoplados de funcionalidades más pequeñas para no

Tema 3. Principios SOLID

verse afectados por ellas y por sus cambios.

Este principio se basa en la abstracción, las implementaciones concretas no deberían depender de otras implementaciones concretas si no de capas abstractas.

Este principio nos permite, por ejemplo, en no depender de si nuestra base de datos esta realizada en una tecnología u otra, porque nuestro código no depende de que base de datos usamos sino de una abstracción que hemos construido entre medias.

Es decir, podemos tener una clase que nos habilite el método getData pero que este método pueda ser por ejemplo una consulta MySQL o una consulta a MongoDB el método de obtener datos será el mismo, y seguramente siga el mismo modelo de datos, el único cambio que habría que realizar para cambiar de BBDD sería cambiar la clase de las consultas el resto de mi aplicación de debería verse afectada.

Veamos un ejemplo.

```
class ConnectDb {
  getData() {
    let data = MongoDB.find({});
    return data
  }
}

class ConnectDb {
  getData() {
    let data = SQLite.query('SELECT * FROM data');
    return data
  }
}

class Controller {
  //no me hace falta saber que base de datos es la que tengo por detras
  implemente llamo al metodo
  constructor() {
    this.data = ConnectDb.getData();
  }
  printData(){
    for(let oneData of this.data)
    {
      console.log(oneData)
    }
  }
}
```

Tema 3. Principios SOLID

```
}
```

En el ejemplo que os pongo aquí se repita a modo de concepto la clase ConnectDb.

Dicha clase cambia en función de la BBDD que estamos usando, pero sigue teniendo los mismos métodos de getData con lo que nuestros Controlador no se ve afectado por el cambio y sigue funcionando de la misma manera, es abstracto al tipo de acción que se realiza dentro de la clase ConnectDb.

Por poner un ejemplo muy trivial sería lo mismo que cuando tú te sientas en el sofá coges el mando de la tele y le das al botón rojo, sabes que esta se va a encender, pero no sabes lo que está ocurriendo por dentro de la tele para que eso ocurra y que seguramente variará dependiendo de la televisión con la que trabajes.

3.3. Finalidad de los principios SOLID

Algo para tener en cuenta con los principios SOLID es que son una herramienta de trabajo cuyo objetivo es ayudarnos a escribir mejor código y de una forma más estructurada y limpia.

Debemos tener esto claro y evitar caer en aplicar estos principios como si fuera la única forma de realizar una codificación correcta, estos principios marcan unas guías no son un patrón que debamos seguir a rajatabla y evaluar qué ventajas e inconvenientes tienen para nosotros el seguir estos principios en el desarrollo en el que nos encontremos.

Debemos analizar por qué y cómo usar dichos principios en nuestro desarrollo para que tengan sentido y realmente sean una ayuda.

Si usar estos principios te lleva a complicar mucho tu sistema tal vez solo quieras aplicar algunos de los principios y ser un poco más flexible en otros aspectos de tu código, como todo dependerá no solo de ti si no del grupo o empresa de trabajo donde te encuentres.

Otro punto que debéis tener en cuenta sobre estos principios es que son transparentes a la tecnología donde se aplican, es decir que no están creados para javascript, java o C, esto quiere decir que las formas de resolverlos o buscar soluciones a sus implementaciones cambia mucho desde el punto de vista de cada lenguaje.

Las herramientas que nos aporta este Máster no dejan de ser herramientas nosotros como desarrolladores también tendremos que aprender a valorar cual nos conviene en cada parte de nuestro desarrollo.

4.1. Introducción y objetivos

CI/CD son las siglas de Continuous Integration (CI) and Continuous Deployment (CD). En este capítulo vamos a ver que significa esta siglas y ejemplos de cómo se integran en equipos de trabajo.

El cómo realizar estas dos acciones va a variar mucho dependiendo de la empresa donde nos encontremos, el stack tecnológico que manejemos y como está organizado dentro de nuestro equipo.

Por eso en este tema no vamos a entrar en profundidad si no simplemente a sentar unas bases correctas de cómo se aplica esto en el día a día dentro de un equipo de programación y por qué es algo que deberían estar aplicando muchas empresas hoy en día.

4.2. ¿Qué es CI/CD?

El **CI** o **Continuous Integration (CI)** se refiere a las prácticas que se usan normalmente automatizadas para poder integrar el código de las diferentes personas que está trabajando en el proyecto.

A lo que muchos de vosotros os preguntareis, pero eso es **GIT** y ya lo vimos a principio de curso. Bueno GIT es parte fundamental de este proceso, pero no lo único con lo que se realiza.

En este punto entran a otras herramientas como **DOCKER** o **Kubernetes** que, aunque no hemos tratado dentro de este master son herramientas que nos iremos encontrando dentro de nuestros entornos de trabajo y que conviene poco a poco que vayamos conociendo.

Para hacer un pequeño resumen de lo que se hace en **Continuous Integration (CI)** es básicamente accionar los test y herramientas que hemos construido en nuestra aplicación para estar seguros de que nuestro código no entra en conflicto con el código de otra persona, ya sea probándolo en una imagen de DOCKER, en un servidor de desarrollo o en un servidor de test.

Una vez que estemos seguros de que nuestro código no tiene ningún conflicto podemos mergearlo/volcarlo a la rama main sin problema.

Si hubiese algún problema entre tu código y el de otro desarrollador este es el momento para solucionarlo.

Generalmente y lo interesante de esto es que estos procesos se automatizan por lo que hay que configurar los repositorios de código en **GITHUB** o **GITLAB** para que corran estas herramientas de automatización o scripts y permitan realizar cada uno de los pasos para comprobar esto.

Sigo remarcando que la intención de este tema no es ver a fondo dichas herramientas ya que son muchas y algunas incluso de programan a medida si no que entendáis el proceso para que cuando estéis en una empresa donde lo realicen al menos sepáis de que están hablando.

Normalmente el equipo Sistemas o Devops es el que se encarga de configurar todo esto y te proporciona las herramientas para realizarlo.

El CD o Continuous Deployment (CD) en realidad son dos partes **Continuous Delivery and Continuous Deployment** y en algunos equipos de trabajo se definen estas tareas como **Continuous Distribution**.

La parte de **Continuous Delivery** se refiere a todas las acciones que recogen el código que ya está testeado y listo para mergear dentro de los pasos de CI, y lo meten en nuestro repositorio. En este punto se puede hacer con GIT o con DOCKER o con otras muchas herramientas de desarrollo.

Lo que se marca en esta fase es simplemente los pasos necesarios para realizar un paso previo a la salida da producción, sería como sacar una RELEASE de nuestro repositorio para hacer una comprobación previa ya sea por nuestro LEAD o por el cliente final antes de su salida a producción.

La última parte del **CD Continuous Deployment** se encarga de coger ese RELEASE o rama previa al deploy y pasarla a nuestra rama final MAIN o rama de producción.

Básicamente estos automatismos de los que se encargan son de hacer que el desarrollador solo se tenga que encargar de hacer su código o tarea asignada subirlo a un repositorio por medio de GIT, por ejemplo. Y una vez que ese desarrollador hacer «**git push**» se desencadenan una serie de procesos y herramientas que van realizando poco a poco las cosas.

Todas estas herramientas lo que hacen es comprobar que nuestro código no entra en conflicto con nadie que sea limpio, que pase los test designados y si no nos lo

comunique, crear un RELEASE y por último subirlo a producción si todos estos pasos han ido de forma correcta.

En resumen, CI/CD es un flujo de trabajo que cada empresa monta en función de sus necesidades y las tecnologías que maneja, para garantizar a los desarrolladores un conjunto de herramientas que les permitan desplegar su aplicación sin tener que hacer procesos manuales tales como los que hemos explicado en el tema de deployment.

Esto no nos exime a los desarrolladores de conocer los pasos para desplegar una aplicación si no lo que garantiza es que dentro de una empresa donde hay muchos desarrolladores los conflictos sean los menos posibles y podemos centrarnos en trabajar con el código y no tener que estar resolviendo problemas en el deploy de nuestras aplicaciones ya que estos son procesos que se realizan de forma autónoma.

Lo que conseguimos con CI/CD es automatizar a un ordenador para que haga las acciones que se supone que tenemos que hacer nosotros y que como seres humanos nos podemos equivocar. Al realizarlos una máquina conseguimos que esos procesos se realicen siempre de la misma forma.

4.3. Cómo podemos implementar CI/CD

Una de las herramientas que han venido a integrar esto de las tareas automatizadas en **GITHUB ACTIONS**.

Con actions de GITHUB puedes crear un flujo de trabajo de tareas en tu repositorio que puede consistir en:

- ▶ Instalar las dependencias de mi proyecto (NPM).
- ▶ Correr los tests para realizar comprobaciones de nuestro código.
- ▶ Ejecutar el build de mi aplicación para generar el código para hacer el deploy.

De esta forma nuestro repositorio nos avisará mediante una pull request que los test han pasado y que no se ha roto nada en nuestro repositorio.

Estos dos puntos pertenecen a la parte de integración continua, una herramienta como **GITHUB ACTIONS** me permite realizarla de forma más o menos sencilla.

Recuerdo que todos estos procesos son automáticos y que se realizan justo después de que hagamos nuestro «**git push**» al servidor de desarrollo donde trabajamos.

Una vez que hemos pasado los pasos anteriores, toca subir nuestro código liberado a producción o a nuestros servidores de RELEASE.

Esta parte se llama CD o despliegue continuo, y se puede realizar manualmente a través de un servidor FTP una terminal SSH o de forma automatizada.

Lo más probable es que se creen scripts de automatización por parte del personal de sistemas que me permita una vez validados los test y hecho el build desplegarlo dentro de nuestra rama main para su salida a producción o a su salida en RELEASE.

4.4. ¿Por qué implementarla en nuestros proyectos?

Como ya hemos dicho con anterioridad con estos procesos podemos automatizar y tener controlados nuestros procesos de creación de aplicaciones web, desde la integración, construcción, despliegue y administración de nuestras aplicaciones.

Todo esto mediante ciclos de desarrollo reducido, incrementando la frecuencia de despliegues, y por tanto entregando un valor constante al usuario final que recibe las mejoras de forma muy continuada.

En conclusión, la CI/CD incorpora la automatización continua y el control permanente en todo el ciclo de vida de nuestras aplicaciones.

Una imagen que ilustra muy bien este tipo de procesos y que está bastante generalizada en el mundo del desarrollo es:

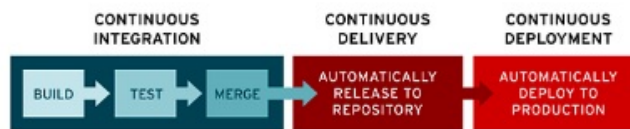


Figura 18. Pasos del CD/CD Fuente. Cortesía de redhat.com

Herramientas para realizar CI/CD

Existen varias herramientas en el mercado que nos ayudan a implementar la integración y el despliegue continuo.

- ▶ GITHUBS ACTION (hemos hablado de ella en el punto anterior).
- ▶ JENKINS.

- ▶ TRAVIS CI.
- ▶ AWS.
- ▶ AZURE Devops.
- ▶ GITLAB.

El usar una u otra va a depender de tu empresa y el equipo de trabajo donde te encuentres.

Te animamos a que investigues por tu cuenta las diferentes herramientas, aunque los procesos que manejan son en gran medida parecidos.

Cómo usar Testing en Angular con Jasmine y Karma

Digital 55. Página web oficial. <https://www.digital55.com/desarrollo-tecnologia/como-usar-testing-angular-jasmine-karma/>

Artículo Interesante de sobre el tema de testing en Angular con diferentes librerías, y probando diferentes tipos de test.

TDD (Test Driven Development) Desarrollo dirigido por pruebas

Software Crafters. Página web oficial. <https://softwarecrafters.io/javascript/tdd-test-driven-development>

Artículo interesante sobre los test unitarios en javascript nativo y como se implementan.

TDD | Test Driven Development en Api REST con Nodejs y Express desde cero

Medium. Página web oficial. <https://develohero.medium.com/test-driven-development-en-una-api-rest-con-nodejs-y-express-desde-cero-6bb3d79dc56e>

Artículo en médium interesante de como implementar el paradigma TDD a un API-REST realizado por Node JS.

CleanCoder

Uncle Bob. Página web oficial. <http://cleancoder.com/products>

Bob Martin fue el precursor de los principios SOLID y en esta página web tiene mucha más información acerca de estos 5 principios y otros puntos interesantes a tener en cuenta.

Solid principles javascript

LogRocket. Página web oficial. <https://blog.logrocket.com/solid-principles-single-responsibility-in-javascript-frameworks>

Como hemos dicho antes la implementación de los principios dependiendo de la tecnología depende mucho de la propia tecnología de y de sus particularidades, hay amplia información en internet acerca de los principios SOLID en javascript, aunque la publicación de esta noticia ayuda a ver su implementación en frameworks como Angular.

S.O.L.I.D The first 5 principles of Object Oriented Design with JavaScript

Medium. Página web oficial. <https://medium.com/@cramirez92/s-o-l-i-d-the-first-5-principles-of-object-oriented-design-with-javascript-790f6ac9b9fa>

Más información acerca de los principios SOLID en Javascript.

Comparación de integración continua, entrega e implementación continuas

Atlassian. Página web oficial. <https://www.atlassian.com/es/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

Atlassian es una compañía que tiene multitud de herramientas muy interesantes en este ámbito, escribió un artículo bastante interesante sobre las diferencias entre integración continua, entrega e implementación continuas. Leerlo puede ayudarte a aclarar algunas cosas sobre este tipo de herramientas.

1. ¿Qué nos aporta el TDD?

- A. Porque es lo que más se usa actualmente en el mundo del desarrollo.
- B. Porque nos ayuda a darnos cuentas de procesos por los que tiene que pasar nuestro software y tenerlos previstos y que cumpla el propósito para el que fue creado.
- C. Escribir test es un ejercicio muy importante para el desarrollador ya que le ayuda a pensar en la posible solución al problema.
- D. Es una forma de controlar que los desarrolladores hacen bien su trabajo y poderlo validar.

2. Los test se ejecutan antes, después o durante el desarrollo

- A. Se desarrollan durante y se ejecutan al final del desarrollo para validarlo.
- B. Es indiferente cuando se realizan, pero se ejecutan siempre al final del desarrollo.
- C. Se suelen realizar antes del desarrollo de dicho módulo porque me permite darme cuenta de las necesidades a resolver y se ejecuta de forma continua durante todo el desarrollo.
- D. Según los principio y reglas del código limpio se tienen que desarrollar los test antes de empezar el desarrollo y probarlos una vez terminado el mismo.

3. Relacionar...

Jasmine	1
Jest	2
Nombre.spec.ts	3
Nombre.spec.js	4

A	Librería de pruebas en Express.
B	Fichero de pruebas en cada componente de Angular.
C	Librería de pruebas en Angular.
D	Fichero de pruebas genérico del Express.

4. En una aplicación de test para backend en Express nuestras pruebas ¿pueden escribir en la base de datos?
- A. No solo pueden si no que es un proceso totalmente normal.
 - B. No, las pruebas en backend solo se crean para validar procesos de la programación en javascript en ningún momento están pensadas para validar los procesos que ocurren dentro de la una BBDD.
 - C. Depende del proyecto y las circunstancias, deberías implementar los test también en las BBDD incluso en el lenguaje propio del sistema de BBDD como por ejemplo SQL.
5. ¿Cuáles de estas opciones son beneficios del TDD?
- A. El TDD genera un código tremendamente optimizado.
 - B. Todas.
 - C. Nos permiten darnos cuenta de las necesidades que va a tener el cliente antes de ponernos a desarrollar.
6. ¿Cuáles son los principales objetivos que intentamos cumplir con SOLID?
- A. Que nuestro código sea más mantenible.
 - B. Todas.
 - C. Facilitarnos implementar nuevos desarrollos.
 - D. Que tengamos un código limpio y fácil de entender.

7. SOLID se debe usar siempre en nuestros desarrollos dentro de cualquier lenguaje

- A. Por supuesto, son normas de desarrollo que todo el mundo debería seguir
- B. No simplemente son patrones que nos aconsejan seguir, pero en ningún caso son obligatorios, son pautas que alguien se inventó y que funcionan.
- C. Son puntos importantes a tener en cuenta, pero su implementación no es una norma simplemente consejos que deberíamos seguir. La aplicación de uno a más principios SOLID depende de muchos factores y es el desarrollador y el grupo de trabajo el que debe decidir implementarlo.

8. Relaciona cada principio con su explicación:

Single Responsibility	1	A	«Todas las entidades de software deben quedarse abiertas para su posible extensión, pero cerradas a modificación».
Open Closed	2	B	«Nunca debería haber más de un motivo por el cual cambiar una clase o un módulo».
Liskov Substitution	3	C	«Divide y vencerás»
Interface Segregation	4	D	«Las funciones que utilicen referencias a otras clases padre o base deben ser capaces de usar objetos de clases derivadas sin saberlo»
Dependency Inversion	5	F	«Las módulos de alto nivel no deberían depender de los módulos de bajo nivel»

9. ¿CD/CI es un sistema cerrado que se implementa siempre igual?
- A. Para nada, depende mucho de la empresa y de las herramientas que se usen para implementar.
 - B. Si, es una forma de trabajar para facilitar el despliegue de aplicaciones y siempre se desarrolla de la misma forma, simplemente hay que aprender aplicarlo dentro de la empresa donde nos encontremos.
 - C. Depende mucho de los compañeros con los que trabajes ya que es un sistema que tenéis que acordar entre todos.
10. ¿Cuáles de estas herramientas se usan en CD/CI?
- A. Todas ellas.
 - B. GITHUB Actions.
 - C. JENKINS.
 - D. GITLAB.